

Artifact Appendix for KubePACS: Kubernetes Cluster Using Performant, Highly Available, and Cost Efficient Spot Instances

Taeyoon Kim*
tykim7@hanyang.ac.kr
Dept. of Data Science
Hanyang University
Seoul, Republic of Korea

Kyumin Kim*
okkimok123@hanyang.ac.kr
Dept. of Data Science
Hanyang University
Seoul, Republic of Korea

Enrique Molina-Giménez
enrique.molina@urv.cat
Dept. of Computer Eng. and Math.
Universitat Rovira i Virgili
Tarragona, Spain

Pedro García-López
pedro.garcia@urv.cat
Dept. of Computer Eng. and Math.
Universitat Rovira i Virgili
Tarragona, Spain

Kyungyong Lee†
kyungyong@hanyang.ac.kr
Dept. of Data Science
Hanyang University
Seoul, Republic of Korea

ABSTRACT

This artifact accompanies KubePACS, a Kubernetes-native spot-instance provisioning system that jointly considers hardware performance, spot pricing, and multi-node availability. It provides the optimizer, Kubernetes integration sources, stored experimental results, and scripts for generating the paper’s figures and tables. To minimize cloud cost and setup effort, evaluation runs the optimizer on stored inputs and generates figures and tables from the supplied results locally. We request the *Artifacts Available*, *Artifacts Functional*, and *Results Reproduced* badges.

1 ARTIFACT CHECK-LIST

- **Programs:** Python analysis and result-generation scripts with accompanying optimizer and Kubernetes integration sources.
- **Software:** Python 3.11 and `uv` for figures, using the supplied lock file. The optimizer check adds PuLP 2.9.0 and requests 2.32.5. Table generation uses the Python standard library.
- **Run-time environment:** Linux workstation for local analysis. Internet access for repository retrieval and dependency installation.
- **Hardware:** General-purpose CPU. No GPU required for local result generation.
- **Disk space:** Approximately 4 GiB for the local figure workflow, including dependencies, temporary inputs, and generated outputs.
- **Execution time:** Allow several minutes for all figures. Each plotting script has a 180-second timeout. Allow several additional minutes for the optimizer check.
- **Outputs:** 24 figure PDFs and Tables 2 and 3 in CSV, Markdown, and LaTeX. Optimizer recommendations are saved as ten JSON reports. Figure execution logs and a machine-readable summary are also provided.
- **Access:** Source, data, and instructions at <https://github.com/ddps-lab/KubePACS>.

*Both authors contributed equally to this work.

†Corresponding author.

2 DESCRIPTION

2.1 Access and Structure

The repository is the entry point for installation and evaluation. Commands in this appendix start at its root.

```
KubePACS/  
|-- README.md  
|-- optimizer/  
|   |-- check_optimizer.py  
|   |-- scenarios.json  
|   +-- data/  
|-- figures/  
|   |-- reproduce.py  
|   |-- figure*/  
|   |-- table2_alpha/  
|   +-- table3_compute/  
+-- api/
```

2.2 Software and Data

Optimizer and integration. The optimizer combines integer linear programming with Golden Section Search to select instance types, availability zones, and node counts. The `api/` directory exposes this implementation through a Lambda handler that can also be invoked locally. Kubernetes integration sources are also supplied.

Experimental inputs. The `figures/` directory contains benchmark data, stored simulation and application results, plotting scripts, and reference PDFs. Historical price inputs are supplied locally, so figure generation does not query live market data. Table 2 aggregates allocation and alpha-sweep results, including the supplied $\alpha=1$ records. Table 3 uses the recorded compilation and video encoding throughput and instance prices. Figures 3 and 4 are architectural illustrations supplied as PPTX sources. Table 1 defines mathematical notation.

3 RELATIONSHIP TO THE PAPER

The artifact connects the paper’s claims to implementations and stored results as follows. Figure and table numbers below refer to the main paper.

- **Cost–performance optimization:** benchmark and price comparisons and baseline evaluations in Figures 1, 5, and 10.
- **Multi-node availability:** recorded request fulfillment and availability indicators in Figures 2 and 9.
- **Adaptive optimization:** alpha sensitivity, search overhead, and configuration comparisons in Figures 6–7 and Table 2.
- **Workload-aware selection:** specialized instance selection in Figure 8.
- **Application efficiency:** recorded infrastructure metrics, graph-analytics execution times, and compute-workload comparisons in Figures 10–11 and Table 3.

The local scripts generate Figures 1, 2, and 5–12 and Tables 2–3 from the supplied experimental results.

4 INSTALLATION AND EXECUTION

4.1 Local Figure and Table Generation

Install Git and curl on Linux. Install uv and Python 3.11, then obtain the repository and run:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
uv python install 3.11
git clone https://github.com/ddps-lab/KubePACS.git
cd KubePACS
uv sync --locked --project figures --python 3.11
uv run --locked --project figures \
  python figures/reproduce.py
uv run --locked --project figures \
  python figures/table2_alpha/table_02.py
uv run --locked --project figures \
  python figures/table3_compute/table_03.py
```

Dependency installation requires Internet access. The generation commands then operate on local inputs. Choose a new `--output` directory for subsequent runs.

Data processing and collection. The figure runner executes 16 scripts in a temporary input copy, leaving reference files unchanged. It collects PDFs, per-script logs, and `summary.json` under `artifact-results/figures/`. Table outputs are collected under `artifact-results/table2/` and `artifact-results/table3/`. The table commands emit `table2.csv/.md/.tex` and `table3.csv/.md/.tex`. Table 2 also emits `table2_details.csv` with 1,200 normalized observations.

Expected outputs. The figure command exits with code 0 and reports `PASS: 16 scripts, 24 PDFs`. The summary lists script exit codes and output-presence checks. The table commands report `PASS: Table 2` and `PASS: Table 3`. Table 2 checks all five rounded values against the paper: 0.8616, 0.9563, 0.0006, 0.0001, and 1.0000 for Greedy, $\alpha=0$, $\alpha=0.5$, $\alpha=1$, and GSS, respectively.

Interpretation. Compare generated plots with the supplied references using the metric definitions and figure-by-figure mapping in `figures/README.md`. Table 2 normalizes each configuration’s efficiency to GSS for the same run and scenario, then averages 240 ratios per configuration. Table 3 summarizes the two compute workloads and their cost and throughput comparisons. The local workflow reprocesses

stored experimental results for generation and inspection of the figures and tables.

4.2 Local Optimizer Check

The `optimizer/` directory provides an independent locked environment and regional CSV and AZ mapping files. Run:

```
uv sync --locked --project optimizer
uv run --locked --project optimizer \
  python optimizer/check_optimizer.py
```

The ten scenarios in `optimizer/scenarios.json` vary pod count, CPU, and memory across Virginia, Oregon, Ireland, and Tokyo. They use stored prices, availability limits, hardware specifications, benchmark scores, and zone mappings with the existing optimizer implementation.

Use repeated `--case REGION,PODS,VCPU,GIB` arguments to supply different requirements and `--output` to select a fresh output directory. Success is `PASS: 10 local optimizer scenarios`, ten scenario JSON reports, and `summary.json` under `artifact-results/optimizer/`. Each report contains instance types, zone names, instance counts, estimated hourly cost, performance score, selected alpha, and total pod capacity. The command checks that capacity covers the requested pod count and instance counts respect the stored availability limits. It also recomputes cost and capacity from the regional CSV. Compare recommendations across regions and resource requirements.